

Lecture 02

Hash Tables

Hash Table ADT

- The *hash table* is a table of elements that have keys
- A *hash function* is used for locating the position of a key in the table
- A search for an element can be done in $\Theta(1)$ time.

Selected Hash Table ADT Operations

- *insert*, to insert an element into a table
- *retrieve*, to retrieve an element from the table
- an operation to empty out the hash table

Hash Functions

- The input into a hash function is a key value
- The output from a hash function is an index of an array (hash table) where the object containing the key is located
- Example of a hash function:
$$h(k) = k \% \text{table_size}$$

Example Using a Hash Function

- Suppose our hash function is:
$$h(k) = k \% 100$$
- We wish to search for the object containing key value 214
- k is set to 214 in the hash function
- The result is 14
- The object containing key value 214 is stored at index 14 of the array (hash table)
- The search is done in $\Theta(1)$ time

Inserting an Element

- An element is inserted into a hash table using the same hash function
$$h(k) = k \% 100$$
- To find where an element is to be inserted, use the hash function on its key
- If the key value is 214, the object is to be stored in index 14 of the array
- Insertion is done in $\Theta(1)$ time

Consider the Big Picture ...

- If we have one million key values
- Linear search takes $O(n)$ to search in a regular array or a linked list
 - On average, we might compare 500,000 key values
- Binary search algorithm gives $O(\lg n)$
 - $\lg 500000 \approx 19$
- Number of comparison:
 - 1 (hashtable) vs. 19 (binary search) vs. 500,000 (linear search)

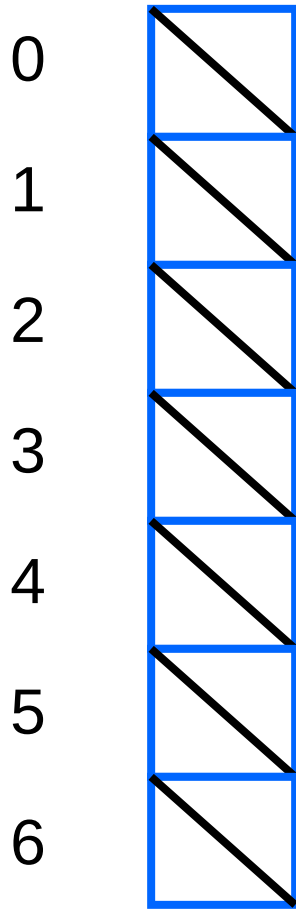
Collisions

- Consider the hash function
 - $h(k) = k \% 100$
- A key value of 393 is used for an object, and the object is stored at index 93
- Then a key value of 193 is used for a second object; the result of the hash function is 93, but index 93 is already occupied
- This is called a **collision**
- The most popular way to resolve collisions is by **chaining, linear probing, and other methods**

Collision Resolution: Chaining

- In chaining, instead of having an array of objects, we have an array of linked lists, each node of which contains an object
- An element is still inserted by using the hash function -- the hash function provides an index of a linked list, and the element is inserted at the front of that (usually short) linked list
- When searching for an element, the hash function is used to get the correct linked list, then the linked list is searched for the key
- Java's HashSet and HashMap are hash tables that use chaining for collision resolution

Example Using Chaining



A hash table which is initially empty.

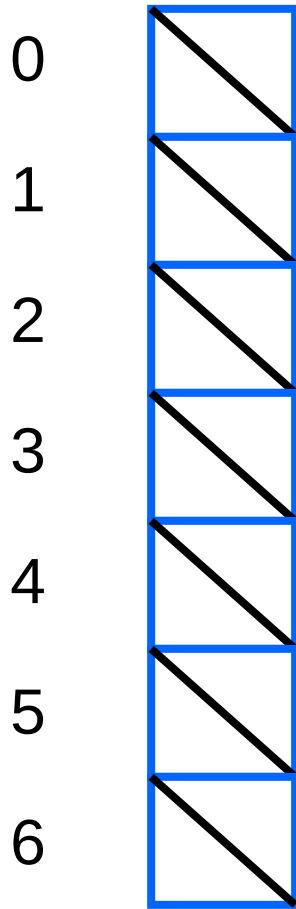
Every element is a LinkedList object. Only the start pointer of the LinkedList object is shown, which is set to NULL.

The hash function is:

$$h(k) = k \% 7$$

Table/Array size is 7

Example Using Chaining (cont.)



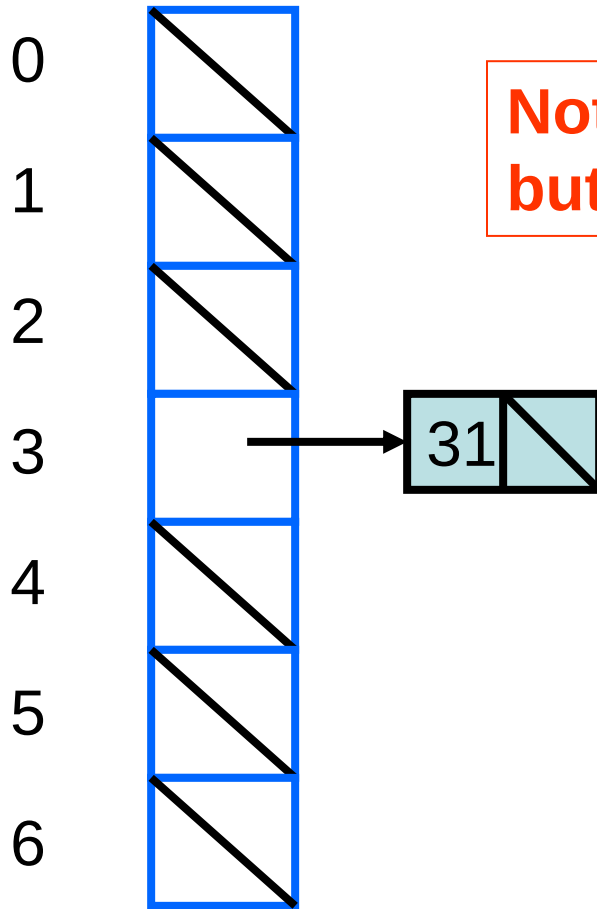
**INSERT object
with key 31**

31 % 7 is 3

The hash function is:

$$h(k) = k \% 7$$

Example Using Chaining (cont.)



**Note: The whole object is stored
but only the key value is shown**

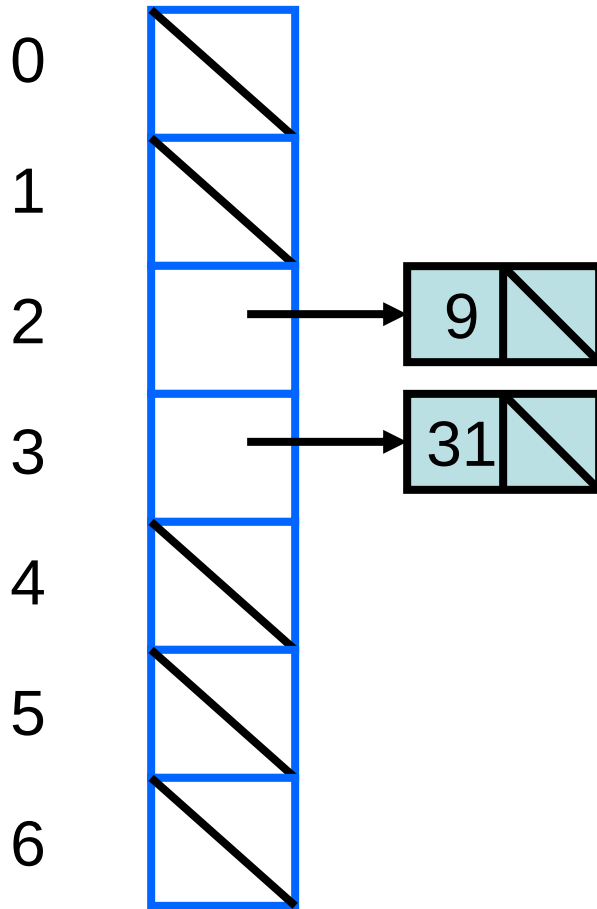
**INSERT object
with key 31**

31 % 7 is 3

The hash function is:

$$h(k) = k \% 7$$

Example Using Chaining (cont.)



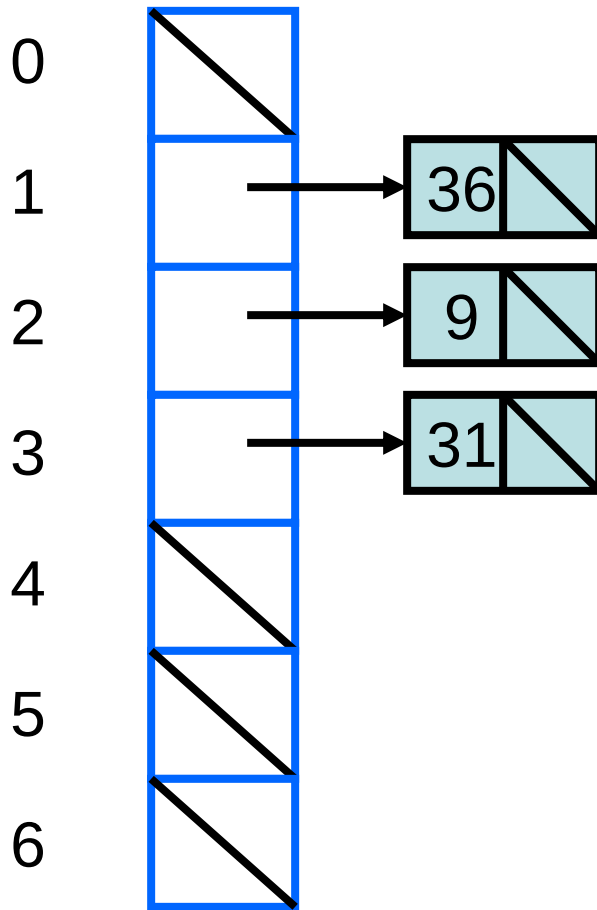
**INSERT object
with key 9**

$9 \% 7$ is 2

The hash function is:

$$h(k) = k \% 7$$

Example Using Chaining (cont.)



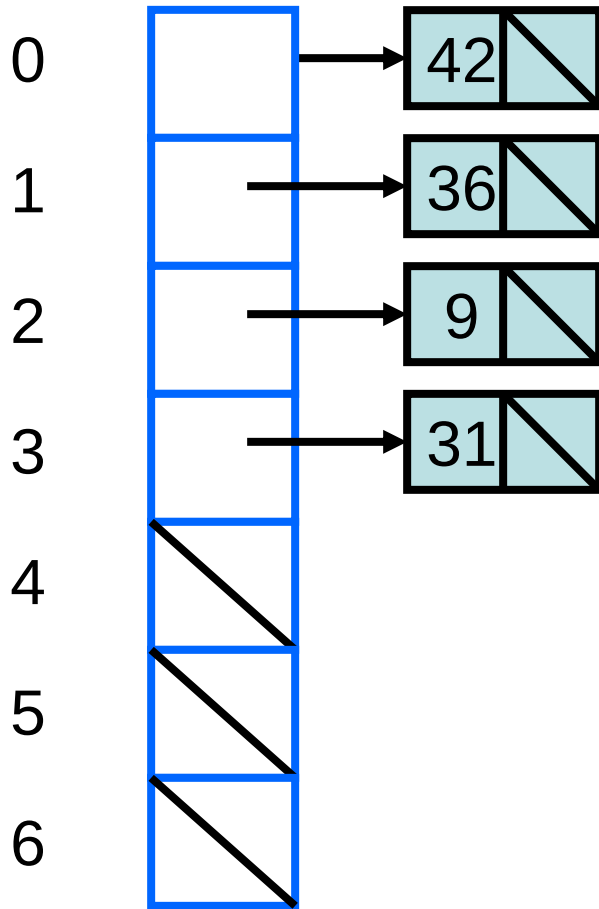
**INSERT object
with key 36**

$36 \% 7$ is 1

The hash function is:

$$h(k) = k \% 7$$

Example Using Chaining (cont.)



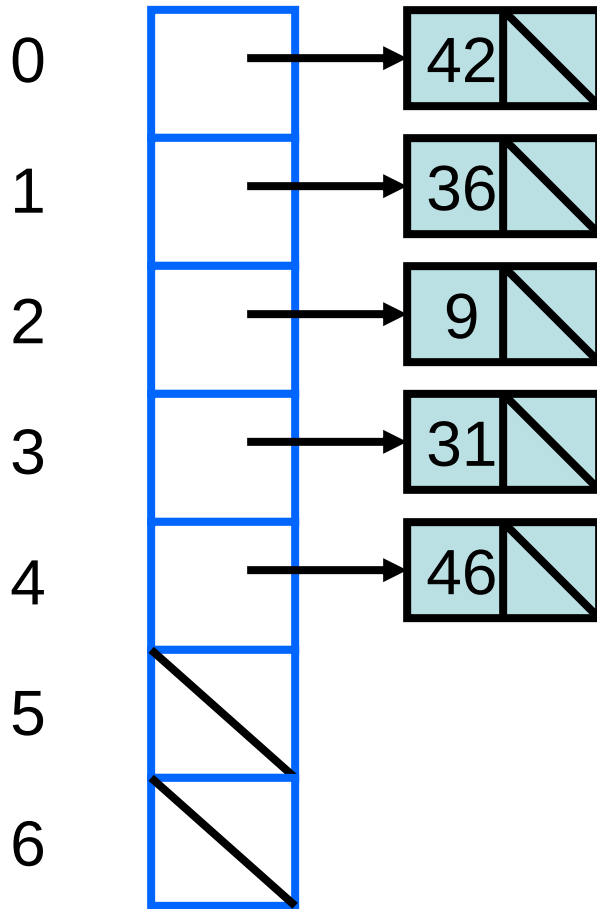
**INSERT object
with key 42**

$42 \% 7$ is 0

The hash function is:

$$h(k) = k \% 7$$

Example Using Chaining (cont.)



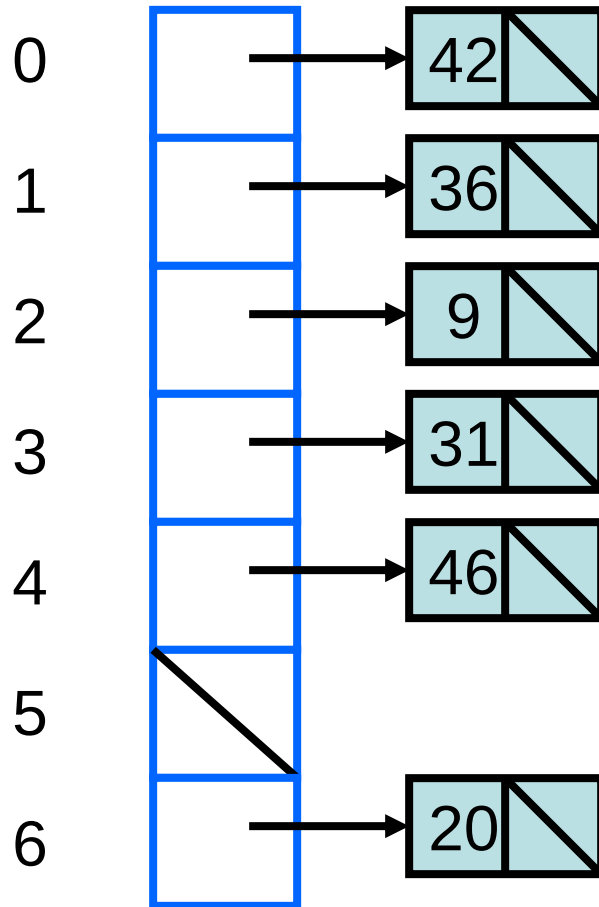
**INSERT object
with key 46**

46 % 7 is 4

The hash function is:

$$h(k) = k \% 7$$

Example Using Chaining (cont.)



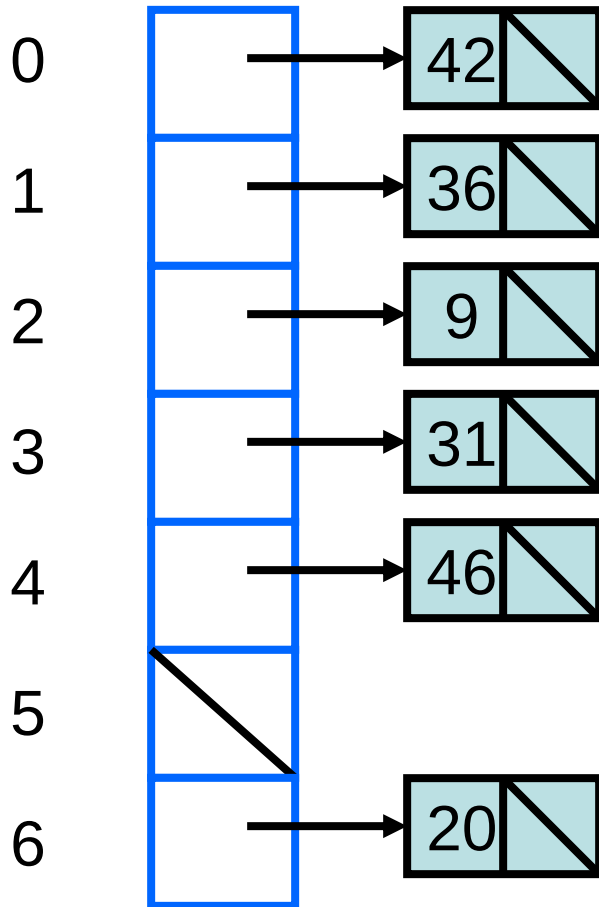
**INSERT object
with key 20**

20 % 7 is 6

The hash function is:

$$h(k) = k \% 7$$

Example Using Chaining (cont.)



COLLISION occurs...

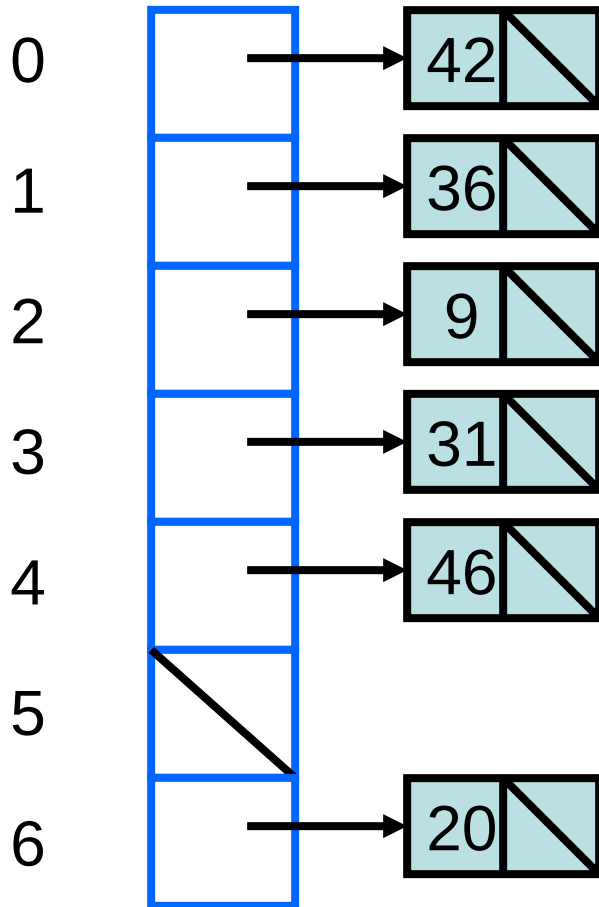
**INSERT object
with key 2**

$2 \% 7$ is 2

The hash function is:

$$h(k) = k \% 7$$

Example Using Chaining (cont.)



But key 2 is just inserted in the linked list

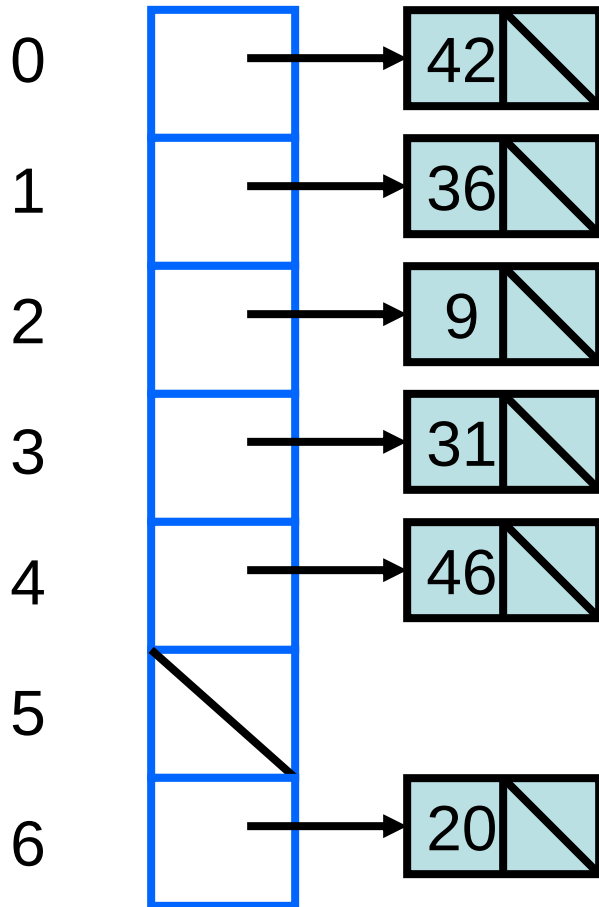
INSERT object with key 2

$2 \% 7$ is 2

The hash function is:

$$h(k) = k \% 7$$

Example Using Chaining (cont.)



The insert function of LinkedList inserts a new element at the BEGINNING of the list

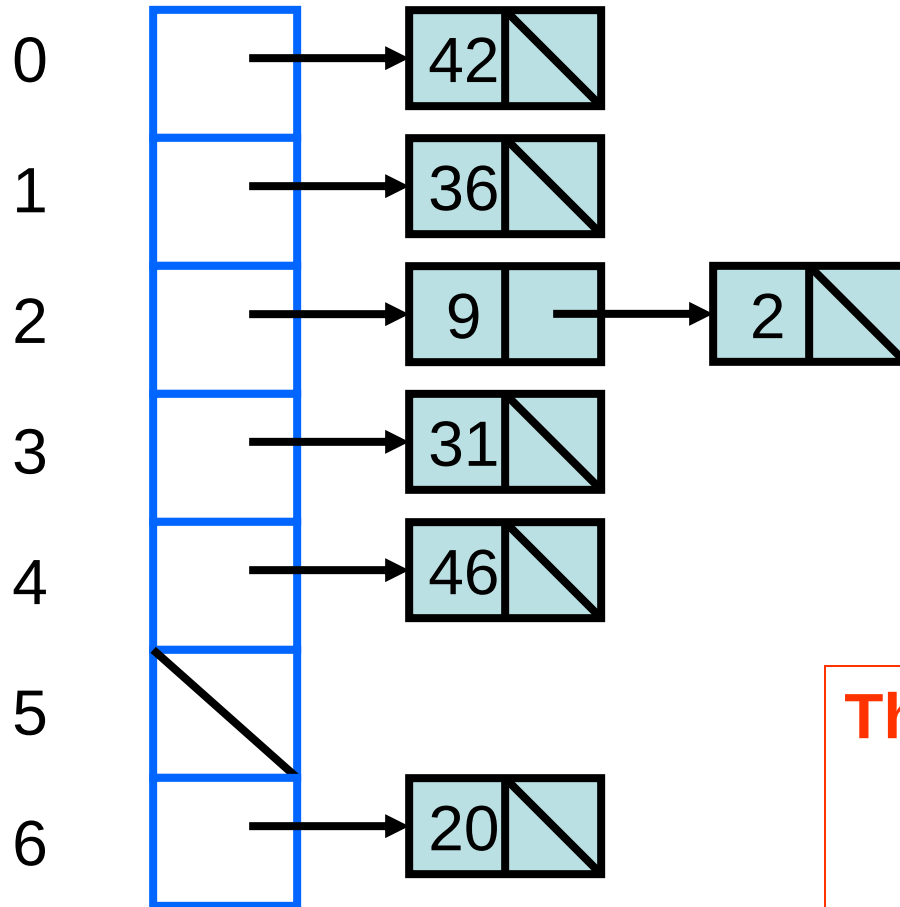
**INSERT object
with key 2**

$2 \% 7$ is 2

The hash function is:

$$h(k) = k \% 7$$

Example Using Chaining (cont.)



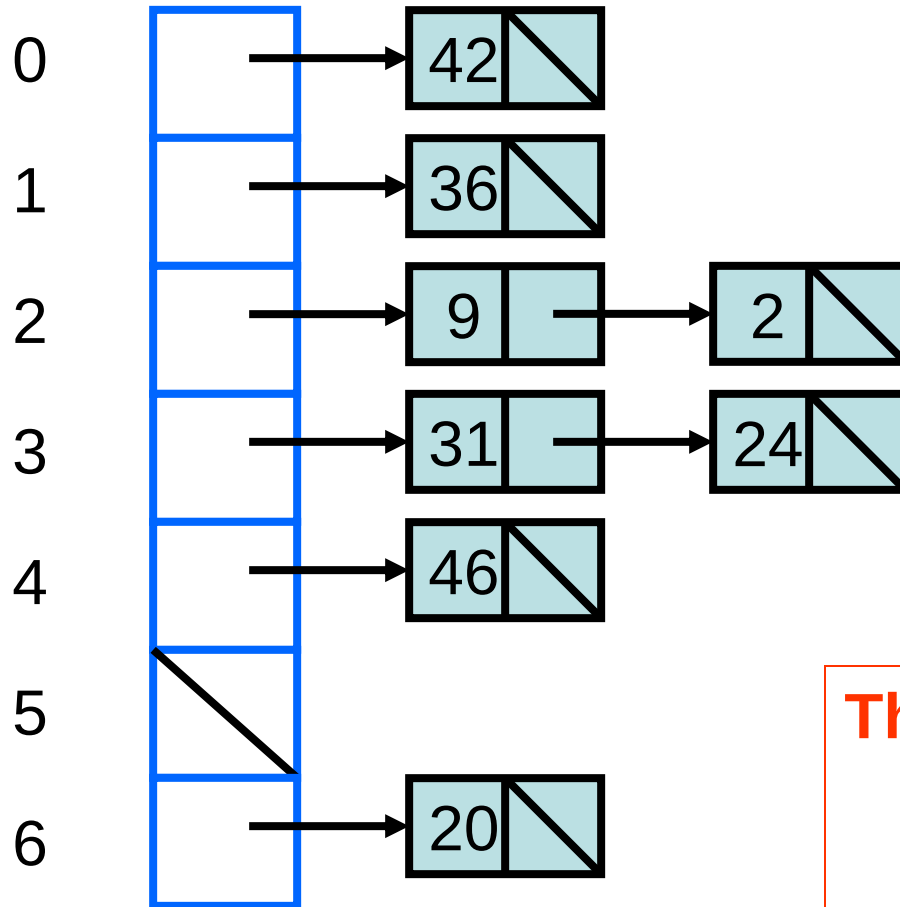
**INSERT object
with key 2**

$2 \% 7$ is 2

The hash function is:

$$h(k) = k \% 7$$

Example Using Chaining (cont.)



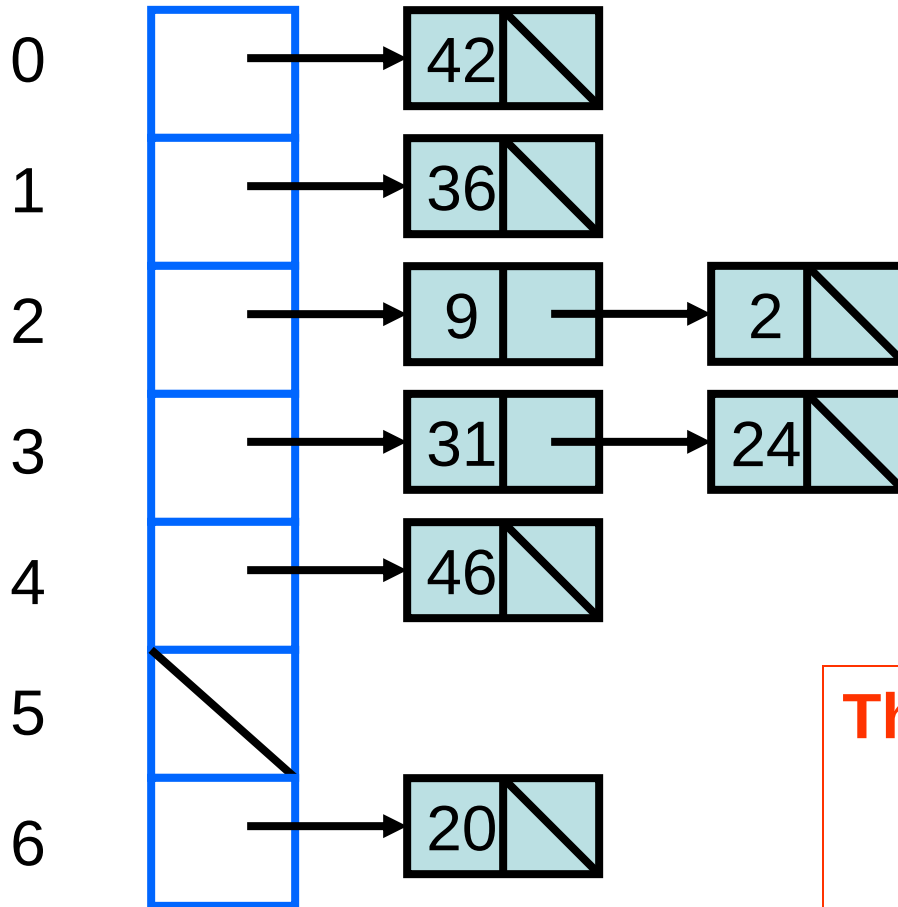
**INSERT object
with key 24**

$24 \% 7$ is 3

The hash function is:

$$h(k) = k \% 7$$

Example Using Chaining (cont.)

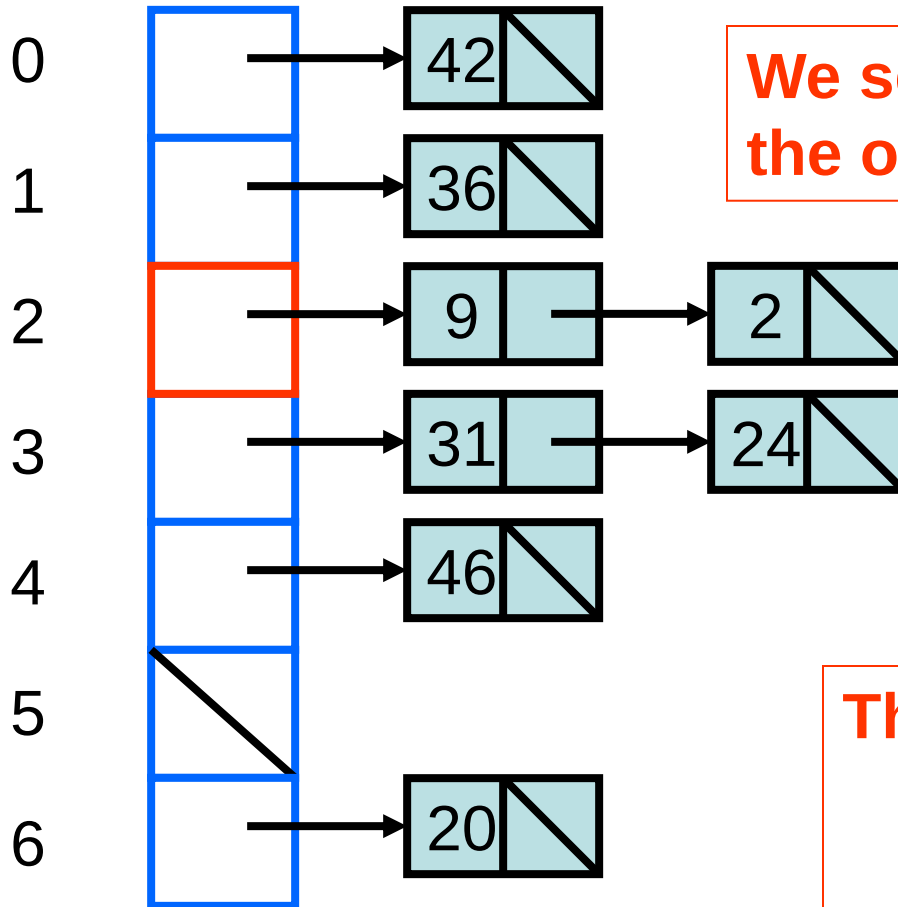


****FIND** the
object with key 2
 $2 \% 7$ is 2**

The hash function is:

$$h(k) = k \% 7$$

Example Using Chaining (cont.)



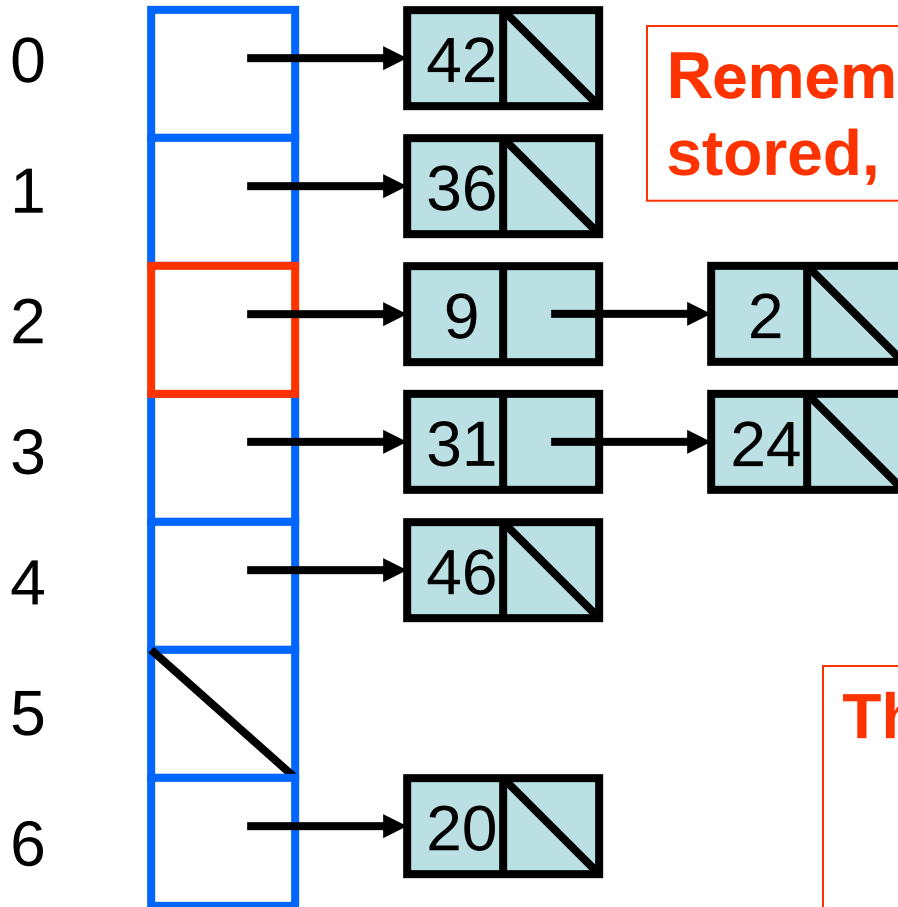
We search this linked list for the object with key 2

****FIND** the object with key 2**
 $2 \% 7$ is 2

The hash function is:

$$h(k) = k \% 7$$

Example Using Chaining (cont.)



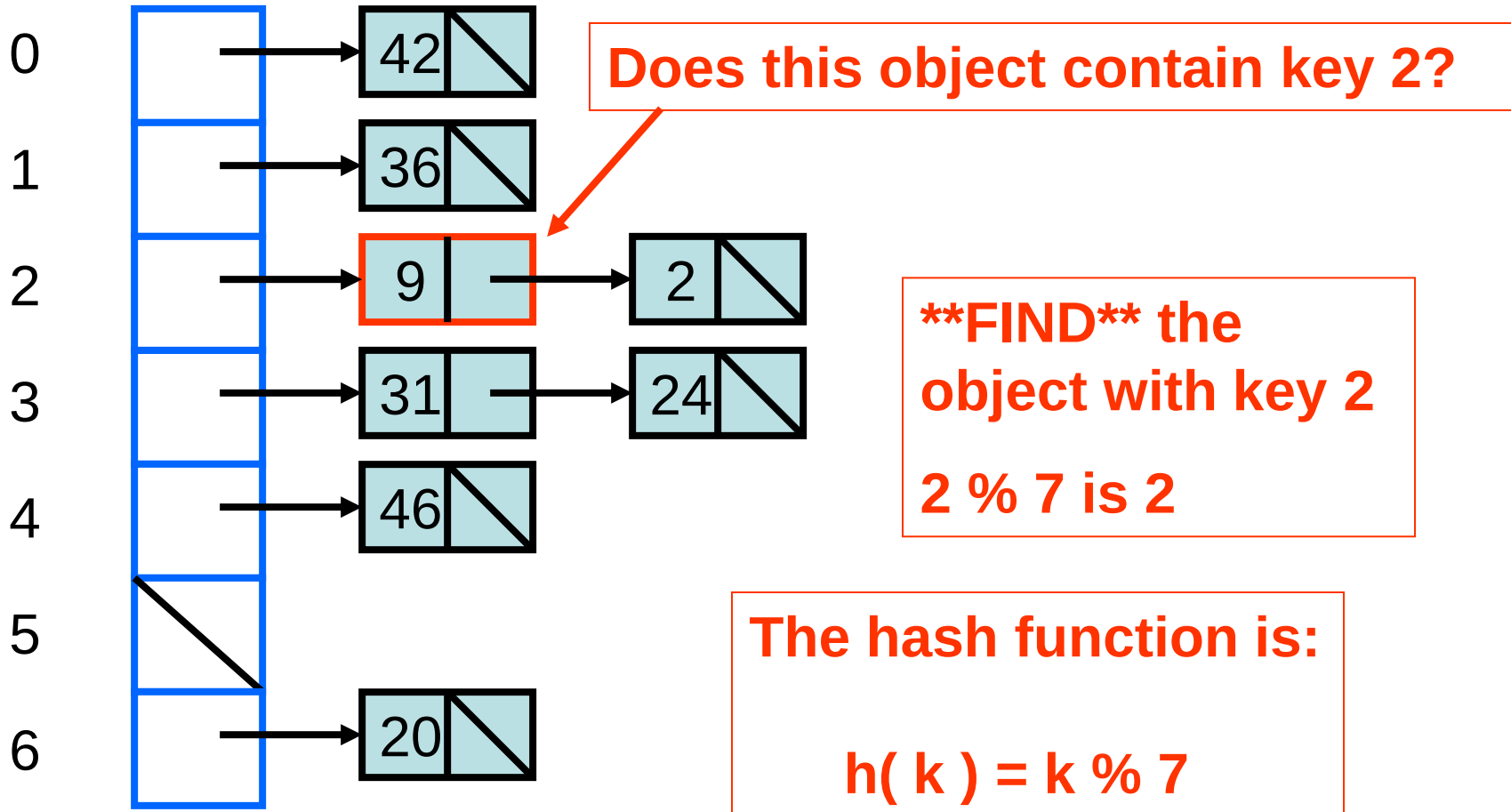
Remember...the whole object is stored, only the key is shown

****FIND** the object with key 2**
 $2 \% 7$ is 2

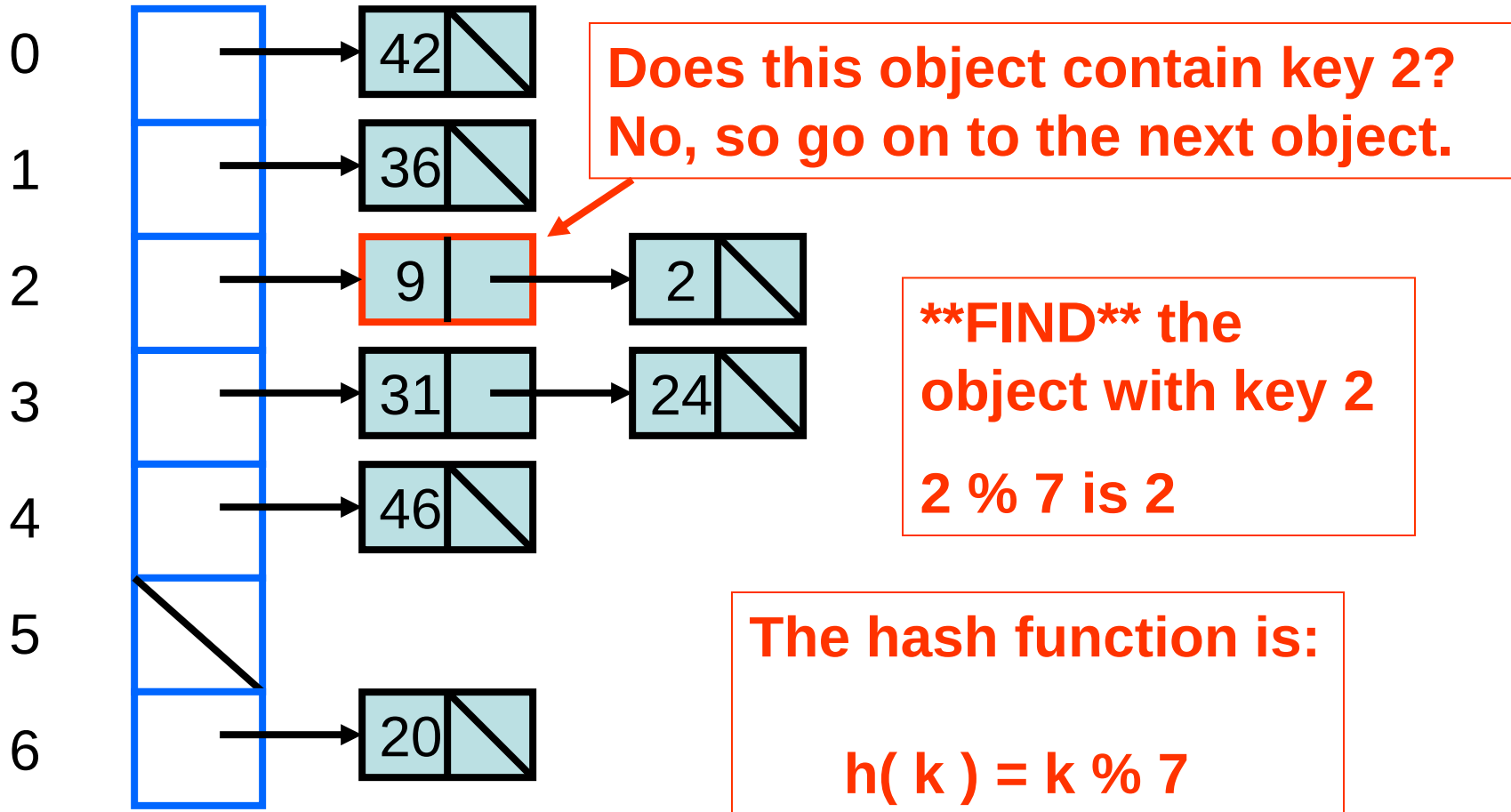
The hash function is:

$$h(k) = k \% 7$$

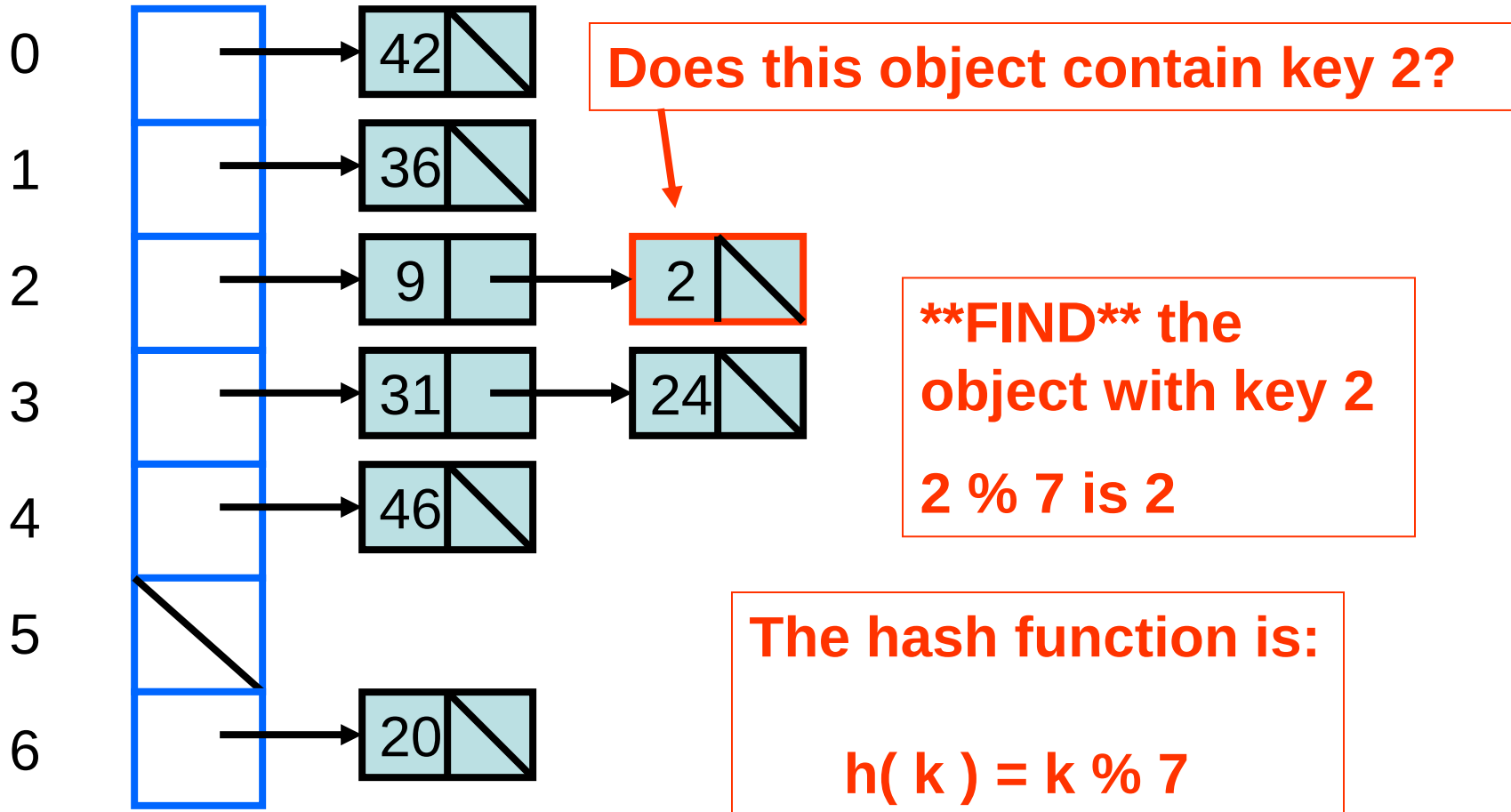
Example Using Chaining (cont.)



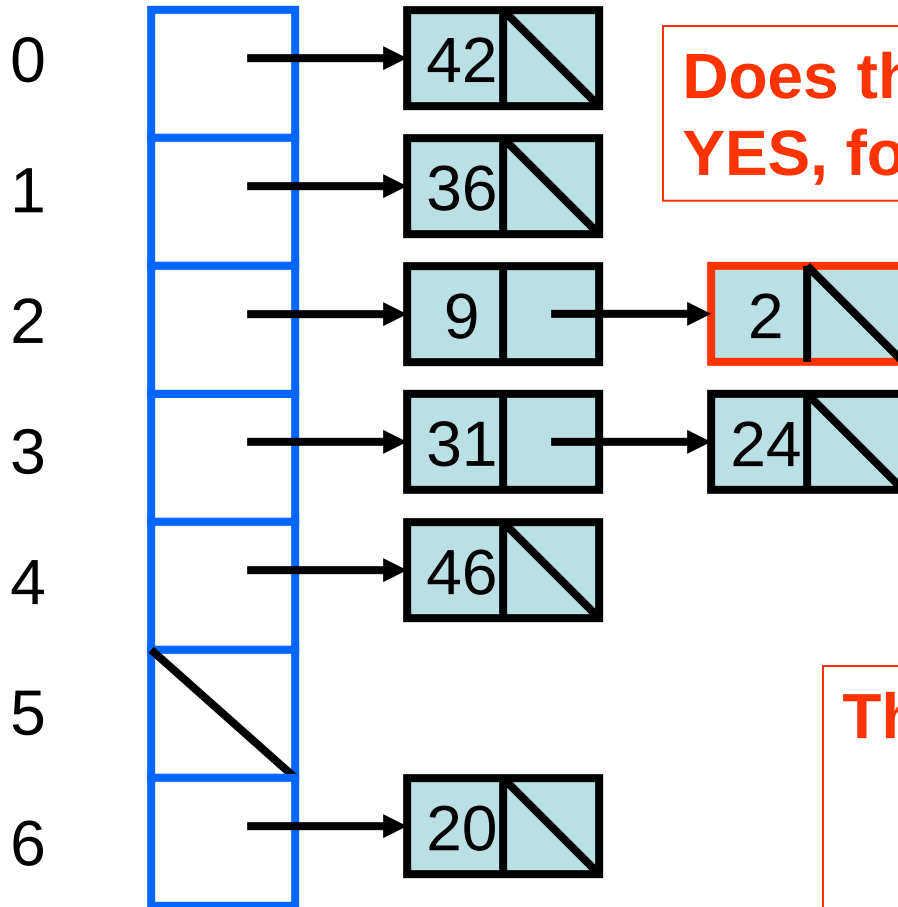
Example Using Chaining (cont.)



Example Using Chaining (cont.)



Example Using Chaining (cont.)



**Does this object contain key 9?
YES, found it! Return the object.**

****FIND** the
object with key 9
 $9 \% 7$ is 2**

The hash function is:

$$h(k) = k \% 7$$

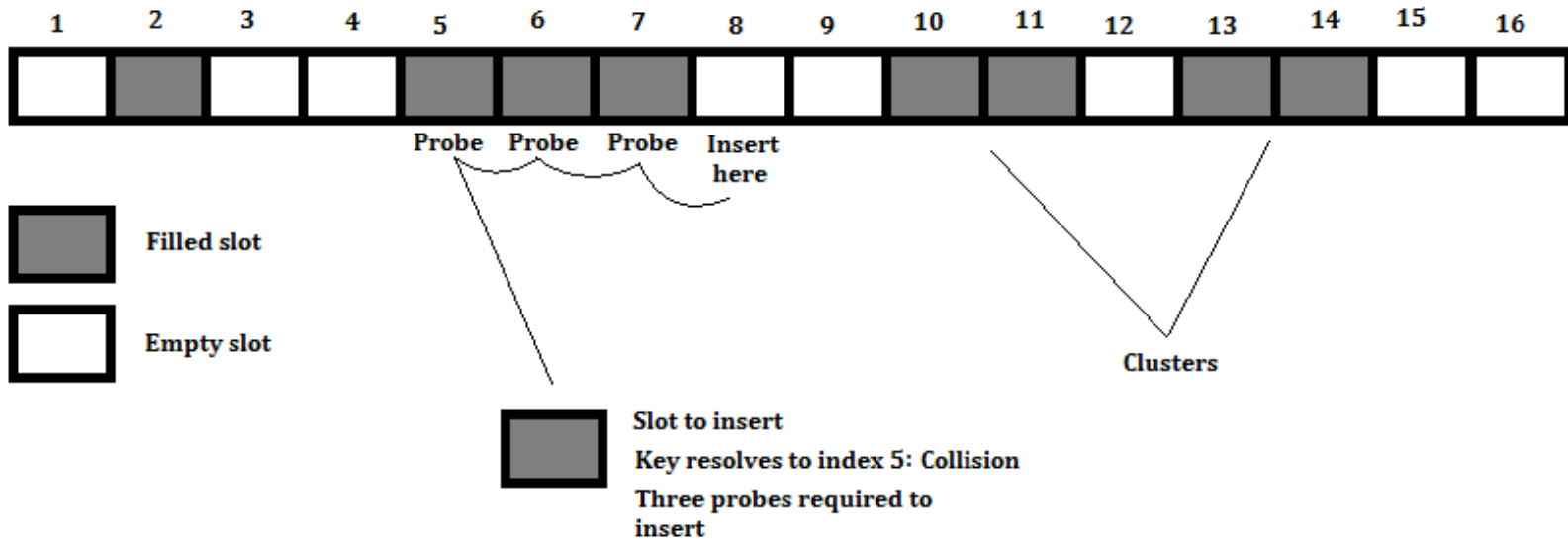
Clustering in Chaining

- Some indexes are crowded but most indexes are empty
- Thus, some linked lists are empty while some linked lists have long chain
- Worst case is $O(n)$ where n is the max length of the chain

Open Addressing

- Chaining requires additional memory for linked list
- If saving memory is important, we could store the elements directly in the array without creating linked list
 - Known as *open addressing*
- Linear probing and quadratic probing are examples of open addressing strategy

Collision Resolution: Linear Probing



- If collision, put the element on the next free slot
- In the above case, collision at index 5, index 6,7 is filled, thus put the item at index 8.

Hashing Linear Probing Problem

HASHING with LINEAR PROBING

$m = 20$, $f(k) = k \% m$

Initial Situation

-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19

After inserting 34, 55, 12, 8, 45, 37, 32, 88, 98, 54

-1	-1	-1	-1	-1	45	-1	-1	8	88	-1	-1	12	32	34	55	54	37	98	-1
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
					0			0	1			0	1	0	0	2	0	0	

After inserting 21, 42, 56, 74, 52, 33, 16

74	21	42	52	33	45	16	-1	8	88	-1	-1	12	32	34	55	54	37	98	56
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
6	0	0	11	11	0	10		0	1			0	1	0	0	2	0	0	3

Red numbers give number of hops in probe sequence

- When inserting 56, we hit index 16, we need to probe till index 19 (3 probes) before we can insert 56.

Clustering in Linear Probing

- Some consecutive indexes are occupied while some consecutive indexes are empty
- Thus, some consecutive array slots are occupied while some consecutive array slots are empty
- Worst case is $O(n)$ where n is the length of the array

Linear Probing improvement

Quadratic Probing

- Rather than always moving one cell (linear probing) when encounters collision, this strategy moves j^2 cells from the point of collision, j is the number of attempts to resolve the collision
- The limitation of this strategy: It may not find an empty cell, if the bucket array is at least half full

Quadratic Probing

$m = 10, f(k) = k \% m$

Initial hashtable:

0	1	2	3	4	5	6	7	8	9

Inserting 27, 17, 37, 47, 48, 57:

$27 \% 10 = 7$

$17 \% 10 = 7$ (Collision: $7 + 1^2 = 8, 8 \% 10 = 8$)

$37 \% 10 = 7$ (Collision: $7 + 2^2 = 11, 11 \% 10 = 1$)

$47 \% 10 = 7$ (Collision: $7 + 3^2 = 16, 16 \% 10 = 6$)

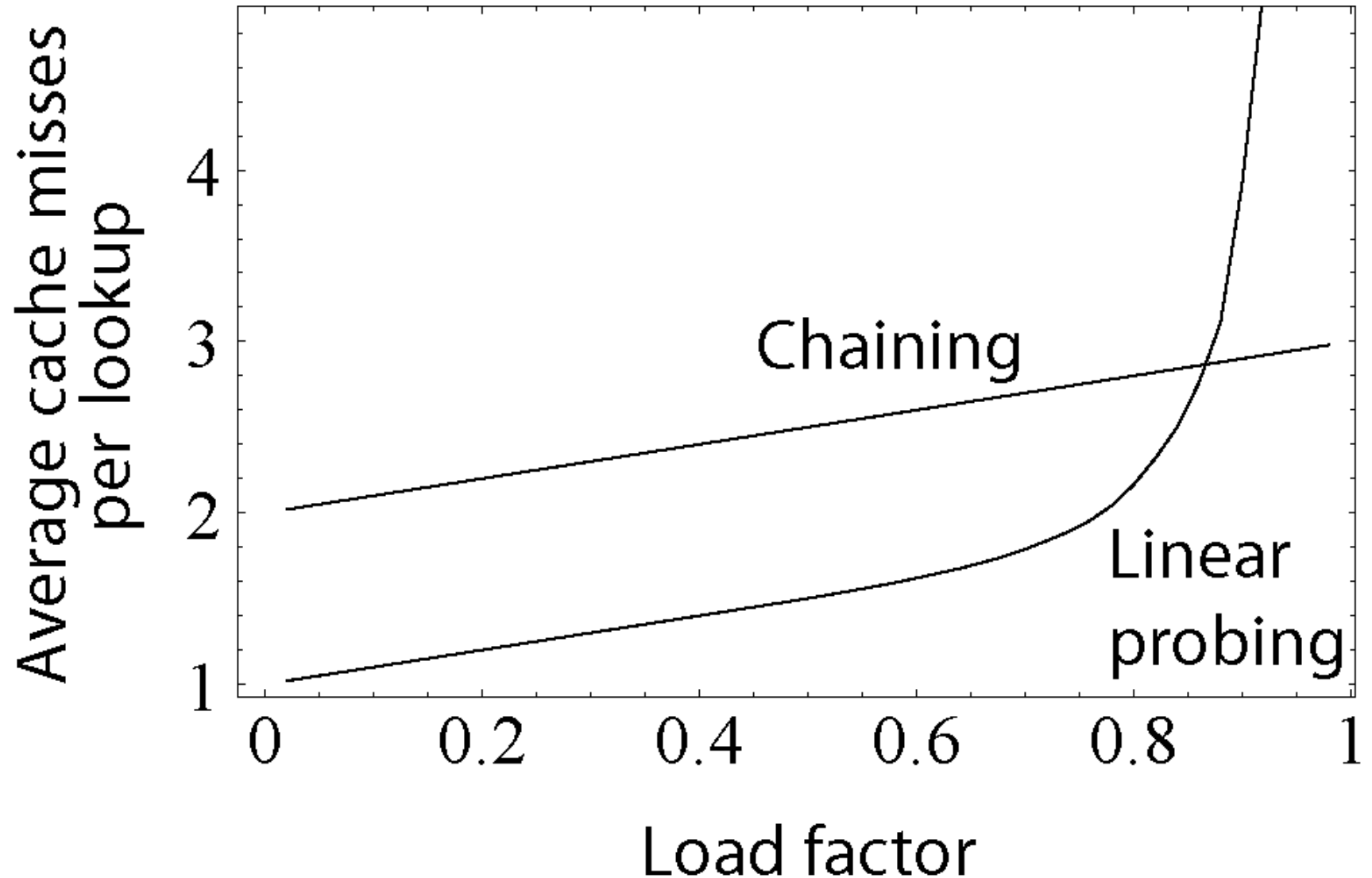
$48 \% 10 = 8$ (Collision: $8 + 1^2 = 9, 9 \% 10 = 9$)

$57 \% 10 = 7$ (Collision: $7 + 4^2 = 23, 23 \% 10 = 3$)

After Insertion:

	37		57			47	27	17	48
0	1	2	3	4	5	6	7	8	9

Chaining vs Linear Probing



Chaining vs. Linear Probing

- Linear Probing uses fixed amount of memory, Chaining requires extra memory for linked lists
- Linear probing is better suited for caching, when you load an item, the next item is always loaded
- Clustering does happen for both techniques, but linear probing search time is longer if there are many collisions
- Linear Probing is better if the load factor is lower than 0.85

Uniform Hashing

- When the elements are spread evenly (or near evenly) among the indexes of a hash table, it is called *uniform hashing*
- If elements are spread evenly, such that the number of elements at an index is less than some small constant, uniform hashing allows $\Theta(1)$ time for a search in both chaining and open addressing, but still $O(n)$ time for a miss in open addressing

Ideal Hash Function for Uniform Hashing

- The hash function largely determines whether we will have uniform hashing
- One possible solution is to have the hash table size a prime number that is not too close to a power of 2
- 31 is a prime number but is too close to a power of 2
- 97 is a prime number not too close to a power of 2
- A good hash function might be:

$$h(k) = k \% 97$$

Ideal Hash Tables

- Can we have a hash function which guarantees that there will be no collisions?
- Yes:
$$h(k) = k$$
- Each key k is unique; therefore, each index produced from $h(k)$ is unique
- Consider 300 employees that have a 4-digit unique id, a hash table size of 10,000 guarantees the best possible speed. However, 97% of the hash table (9,700 slots) are empty hence wasting space

Ideal Hash Tables (cont.)

- Can we have a hash table without any collisions and without any empty slots?
- Sometimes. Consider 300 employees with id from 0 to 299. We can make a hash table size of 300, and use $h(k) = k$

Speed vs. Memory Conservation

- The greatest speed comes about by making a hash table much larger than the number of keys so that there is no collision
- The best memory conservation comes from trying to reduce the number of empty slots

Hash Table Design

- Decide whether speed or memory conservation is more important (and how much more important) for the application
- Come up with a good table size which
 - Allows for the use of a good hash function
 - Strikes the appropriate balance between speed and memory conservation

Time Complexities for Hash Table

- insert – we'll insert at the head of the linked list – $\Theta(1)$
- retrieve – element is found by hashing, so it is $\Theta(1)$ for uniform hashing (the hash function and hash table are designed so that the length of the collision list is bounded by some small constant)